

## Chapitre 1 : Introduction

- Avant les méthodes formelles, la vérification reposait sur :
  - **Les tests (jeux d'essais)** : réalisés sur le programme final.
  - **La simulation** : effectuée sur un modèle théorique avant la réalisation du système.
- **Limites des méthodes classiques** :
  - Correction des erreurs souvent **très coûteuse**.
  - **Non-exhaustivité** des tests.
  - **Non-exhaustivité** des simulations.
  - ⇒ Besoin de **méthodes plus fiables : méthodes formelles**.
- **Objectifs de la vérification** :
  - Assurer la **fiabilité** et la **sûreté** des programmes.
  - Garantir le bon fonctionnement sans erreurs.
  - Éviter toute erreur dans les **systèmes critiques**.
- **Systèmes critiques** :
  - Interagissent en permanence avec leur environnement.
  - Contrôlent un processus (capteurs, signaux, etc.).
  - Ne calculent pas un résultat mais assurent un fonctionnement continu.
  - Appelés **systèmes réactifs**.
  - Souvent **temps réel** et **distribués**.
  - Exemples : aéronautique, téléchirurgie, centrales nucléaires.
- **Exemples de systèmes critiques** :
  - Pacemaker.
  - Pilote automatique.
- **Conséquences d'une mauvaise vérification** :
  - Dégâts matériels.
  - Pertes financières.
  - Pertes humaines.
- **Exemples de catastrophes** :
  - **Ariane V (1996)** :
    - Réutilisation de code non adapté.
    - Erreur de conversion numérique.
    - Défaillance du système de positionnement.
    - Autodestruction de la fusée.
  - **Therac-25** :
    - Sécurité uniquement logicielle.
    - Erreurs de programmation.
    - Surdoses de radiation.
    - Plusieurs morts et retrait des machines.
- **Vérification vs Validation formelle** :
  - **Vérification formelle** :
    - Question : *le programme est-il correct ?*
    - Vérifie la cohérence interne.
    - Utilise des prouveurs de théorèmes (ex. Coq).
  - **Validation formelle** :
    - Question : *le programme respecte-t-il les exigences ?*

- Vérifie la conformité aux spécifications.
  - Utilise l'animation et le model checking.
- **Méthodes de vérification formelle :**
  - Méthodes basées sur la preuve.
  - Méthodes basées sur le modèle (Model Checking).
- **Model Checking :**
  - Représentation du système par un **modèle formel**.
  - Propriétés exprimées en **logique temporelle** (CTL, LTL).
  - Vérification automatique de : **M satisfait  $\Phi$** .
  - Fournit un **contre-exemple** si la propriété est fausse.
- **Étapes du Model Checking :**
  - Définir les propriétés à vérifier.
  - Construire le modèle formel.
  - Lancer la vérification automatique.
  - Analyser le résultat ou le contre-exemple.

## Chapitre 2 : Systèmes de transitions

### 1. Comportement d'un système

- Les systèmes de transitions modélisent le comportement des **systèmes réactifs** et/ou **temps réel**.
- Un système de transition est un **automate**, c'est-à-dire un modèle mathématique de l'évolution dans le temps.
- Utilisés en :
  - Théorie des langages.
  - Modélisation et vérification de systèmes communicants.
- La description du comportement identifie :
  - **Actions** : opérations exécutées par le système.
  - **Événements** : stimuli déclenchant les actions.
  - **Contraintes temporelles** : délais à respecter.
  - **Conditions d'activation** : règles logiques avant l'action.
  - **Situations anormales** : erreurs et exceptions.
  - **États significatifs** : phases importantes du système.
- La modélisation est complexe car :
  - Le nombre de situations peut être infini.
  - Les concepts sont nombreux.
  - Il n'existe pas de modèle universel.

### 2. Automates à États Finis (AEF)

- Un AEF est défini par : **A = (S, E, T, S<sub>0</sub>, F)**.
  - S : ensemble fini d'états.
  - E : alphabet (événements).
  - T : fonction de transition.
  - S<sub>0</sub> : état initial.
  - F : états finaux.
- Représentation :

- États → nœuds.
- Transitions → arcs étiquetés.

### 3. Systèmes de transition

- **STE (Système de Transition Étiqueté) :**
  - $A = (E, Q, q_0, T)$ .
  - Transitions étiquetées.
  - Pas d'états finaux.
  - Modélise uniquement le comportement.
- **ST (Système de Transition non étiqueté) :**
  - $A = (Q, q_0, T)$ .
  - Transitions sans étiquettes.
- Il peut exister **plusieurs états initiaux**.

### 4. États et transitions

#### États

- Représentent une situation du système à un instant donné.
- Abstraction du système.
- Le niveau de détail dépend de la modélisation.

#### Transitions

- Représentent un changement d'état.
- Étiquette possible :
  - Événement.
  - Garde (condition).
  - Actions.
- La forme dépend du type d'automate.

### 5. Types d'automates

- **Déterministe :**
  - Une seule transition possible pour (état, étiquette).
- **Indéterministe :**
  - Plusieurs transitions possibles pour une même étiquette.

### 6. Chemins et exécutions

- **Trace / chemin d'exécution :**
  - Suite d'états ou de transitions.
- **Exécution partielle :**
  - Peut s'arrêter avant la fin.
- **Exécution complète :**
  - Infinie ou bloquée.
- **Arbre d'exécution :**
  - Ensemble de toutes les exécutions possibles.

- Peut être infini.

## 7. Séquences et traces

- $S^*$  : séquences finies.
- $S_\omega$  : séquences infinies.
- **Trace finie** : chemin valide fini.
- **Trace finie maximale** : ne peut plus être prolongée.
- **Trace infinie** : exécution sans fin.

## 8. Atteignabilité

- Un état est atteignable s'il apparaît dans une exécution.
- $Acc(S)$  : ensemble des états accessibles.
- Essentiel pour vérifier les propriétés du système.

## 9. États récurrents

- Un état est récurrent s'il apparaît :
  - Une infinité de fois dans une exécution infinie.
  - En permanence à la fin d'une exécution finie.
- $Inf(\sigma)$  : ensemble des états récurrents.

## 10. Équivalence et structures

- Deux arbres peuvent produire les mêmes traces mais avoir des branchements différents.
- Un STE peut être transformé en ST sans perdre le comportement observable.

## 11. Structure de Kripke

- Extension d'un système de transitions.
- Associe à chaque état les **propriétés vraies**.
- Utilisée en **logique temporelle** et **model checking**.

## 12. Produits de systèmes

- **Produit libre (asynchrone)** :
  - Sous-systèmes évoluent indépendamment.
- **Produit synchronisé** :
  - Certaines actions doivent être exécutées ensemble.

## 13. Propriétés à vérifier

- **Atteignabilité** : une situation peut être atteinte.
- **Sûreté** : rien de mauvais n'arrive.
- **Vivacité** : quelque chose de bon finit par arriver.
- **Équité** : un événement se produit de façon répétée.
- **Absence de blocage** : le système peut toujours progresser.

## Conclusion

- Le comportement d'un système est représenté par des chemins dans un automate.
- Les systèmes de transitions et structures de Kripke sont la base du **model checking**.
- Ils permettent de raisonner formellement sur le comportement et les propriétés des systèmes.

## Chapitre 3 : Logique Temporelle

### 1. Introduction

- La logique temporelle étend la logique classique pour raisonner sur le **temps**.
- Elle permet d'exprimer : *toujours*, *un jour*, *jusqu'à ce que*, sans mesurer le temps explicitement.
- Les propriétés sont vérifiées sur un **modèle** (automate, structure de Kripke).
- Deux logiques principales :
  - **LTL** : raisonnement sur **une seule exécution** (linéaire).
  - **CTL** : raisonnement sur **un arbre d'exécutions**.

### 2. Éléments de base

- **Propositions atomiques** :
  - Décrivent ce qui est vrai dans un état.
- **Connecteurs logiques classiques** :
  - $\neg$ ,  $\wedge$ ,  $\vee$ ,  $\rightarrow$ ,  $\leftrightarrow$
- **Opérateurs temporels** :
  - $X P$  :  $P$  est vrai à l'état suivant.
  - $F P$  :  $P$  sera vrai un jour.
  - $G P$  :  $P$  est toujours vrai.
  - $P U Q$  :  $P$  reste vrai jusqu'à ce que  $Q$  devienne vrai.

### 3. Relations et abréviations

- $G P \equiv \neg F \neg P$
- $F P \equiv \text{True} U P$
- $GF P$  :  $P$  est vrai infiniment souvent.
- $FG P$  : à partir d'un moment,  $P$  est toujours vrai.

### 4. Quantification sur les chemins (CTL)

- **A P** :  $P$  est vrai sur **toutes** les exécutions.
- **E P** :  $P$  est vrai sur **au moins une** exécution.
- Équivalences :
  - $A P \equiv \neg E \neg P$
  - $E P \equiv \neg A \neg P$
- Différence essentielle :
  - $A$  : tous les chemins.
  - $G$  : tous les instants d'un même chemin.

## 5. Opérateurs CTL courants

- **EF P** : P est éventuellement possible.
- **AF P**  $\equiv \neg E \neg F P$  : P arrivera forcément.
- **EG P**  $\equiv E \neg F \neg P$  : P peut rester toujours vrai sur un chemin.
- **AG P**  $\equiv \neg E \neg F \neg P$  : P est toujours vrai sur tous les chemins.
- **EX P** : il existe un état suivant où P est vrai.
- **AX P** : tous les états suivants vérifient P.
- **E[P U Q]** : P jusqu'à Q sur un chemin.
- **A[P U Q]** : P jusqu'à Q sur tous les chemins.

## 6. Transformations importantes

- $EF P = E (\text{True} \cup P)$
- $AF P = A (\text{True} \cup P)$
- $EG P = \neg A (\text{True} \cup \neg P)$
- $AG P = \neg E (\text{True} \cup \neg P)$

## 7. Syntaxe

- Propositions atomiques : p
- Connecteurs booléens :  $\neg \Phi$ ,  $\Phi \wedge \Psi$ ,  $\Phi \vee \Psi$ ,  $\Phi \Rightarrow \Psi$
- Opérateurs temporels :  $X \Phi$ ,  $F \Phi$ ,  $G \Phi$ ,  $\Phi U \Psi$
- Quantificateurs CTL :  $A \Phi$ ,  $E \Phi$
- En CTL, chaque opérateur temporel est précédé de A ou E.

## 8. Sémantique

### 1. Proposition atomique

$$\sigma, i \models p \iff p \in l(\sigma[i])$$

### 2. Négation

$$\sigma, i \models \neg p \iff \text{il n'est pas vrai que } \sigma, i \models p$$

### 3. Conjonction

$$\sigma, i \models p \wedge q \iff \sigma, i \models p \text{ et } \sigma, i \models q$$

### 4. Disjonction

$$\sigma, i \models p \vee q \iff \sigma, i \models p \text{ ou } \sigma, i \models q$$

### 5. Implication

$$\sigma, i \models p \Rightarrow q \iff \text{si } \sigma, i \models p \text{ alors } \sigma, i \models q$$

### 6. Next (X)

$$\sigma, i \models Xp \iff i < |\sigma| \text{ et } \sigma, i+1 \models p$$

### 7. Finally / Eventually (F)

$$\sigma, i \models Fp \iff \exists j, i \leq j \leq |\sigma| \text{ tel que } \sigma, j \models p$$

### 8. Globally / Always (G)

$$\sigma, i \models Gp \iff \forall k, i \leq k \leq |\sigma|, \sigma, k \models p$$

### 9. Until (U)

$$\sigma, i \models p U q \iff \exists j, i \leq j < |\sigma|, \sigma, j \models q \text{ et } \forall k, i \leq k < j, \sigma, k \models p$$

- Abréviations LTL :
  - $F \Phi \equiv \text{True} \cup \Phi$
  - $G \Phi \equiv \neg F \neg \Phi$

## 9. Différence LTL / CTL

- **LTL** :
  - Raisonnement linéaire.
  - Une seule exécution.
- **CTL** :
  - Raisonnement arborescent.
  - Quantification explicite sur les chemins (A / E).

## 10. Propriétés de sûreté (Safety)

- Expriment que *quelque chose de mauvais n'arrive jamais*.
- Formes typiques :
  - $G \neg P$  /  $AG \neg P$  : interdiction.
  - $G P$  /  $AG P$  : invariant.
  - $G (Q \Rightarrow \neg P)$  : sûreté conditionnelle.
  - $G (Q \Rightarrow X P)$  : ordre strict.
- Souvent basées sur **G** ou **AG**.

## 11. Propriétés de vivacité (Liveness)

- Expriment que *quelque chose de bon finira par arriver*.
- Formes typiques :
  - $F P$  /  $EF P$  : atteignabilité.
  - $G (P \Rightarrow F Q)$  : réponse garantie.
  - $GF P$  : événement répété infiniment.
  - $FG P$  : convergence vers un état stable.
- Souvent basées sur **F** ou **EF**.

## 12. Propriétés d'équité

- Garantissent une **justice** dans l'exécution.
- **Équité faible** :
  - Chaque événement finit par arriver au moins une fois.
- **Équité forte** :
  - Chaque événement arrive infiniment souvent.

## 13. Until (ordre des événements)

- **P U Q** :
  - P reste vrai jusqu'à ce que Q arrive.
- **A[P U Q]** :
  - Q arrivera forcément sur tous les chemins.
- **E[P U Q]** :
  - Q arrive sur au moins un chemin.

## Conclusion

- La logique temporelle permet d'exprimer précisément le comportement dynamique des systèmes.
- **LTL** : raisonnement sur une exécution.
- **CTL** : raisonnement sur toutes ou certaines exécutions.
- Elle est essentielle pour formuler et vérifier des propriétés de **sûreté**, **vivacité** et **équité** en model checking.

## Chapitre 4 : Model Checking

- **Model checking** :
  - Méthode de **vérification formelle automatique**.
  - Consiste à vérifier si un **modèle** satisfait une **propriété** :  $A \models \Phi$
  - Méthode **exhaustive** : tous les comportements possibles du système sont pris en compte.
- **Propriétés** :
  - **Existentielles** (ex.  $EF P$ ) :  
→ il suffit de trouver **un seul chemin** où  $P$  devient vrai.
  - **Universelles** (ex.  $AG P$ ) :  
→ il faut montrer que **tous les chemins** satisfont  $P$  en permanence.
- **Problème** :
  - Les chemins peuvent être **très nombreux ou infinis**.
  - Impossible de les tester un par un.
- **Solution** :
  - Utilisation d'**algorithmes automatiques** permettant une vérification en **temps fini**.

## Model Checking de CTL

- En **CTL**, les formules s'appliquent **aux états**.
- Les formules sont :
  - des **propositions atomiques** (et combinaisons booléennes),
  - des formules temporelles : **AG, EG, AF, EF, AX, EX**.
- **Données** :
  - un système  $A$  (structure de Kripke),
  - une propriété  $P$  exprimée en CTL.
- **But** :
  - Déterminer l'**ensemble des états** qui satisfont PPP.
- **Méthode** :
  - Algorithme de **marquage CTL**.
  - Marquage progressif des états satisfaisant chaque **sous-formule**, jusqu'à la formule complète.



## Algorithme de marquage

**Cas  $\phi = p$**  Alors Faire pour tout  $q$  dans  $Q$   
     Faire Si **app** ( $p, q$ ) Alors Faire  $q.\phi := \text{true}$   
     Sinon Faire  $q.\phi := \text{false}$

**Cas  $\phi = \text{not } \psi$**  Alors Faire **marquage** ( $\psi$ );  
     Pour tout  $q$  dans  $Q$  Faire  $q.\phi := \text{non } q.\psi$

**Cas  $\phi = \phi_1 \text{ and } \phi_2$**  Alors Faire **marquage** ( $\phi_1$ );  
     **marquage** ( $\phi_2$ );  
     Pour tout  $q$  dans  $Q$  Faire  $q.\phi := q.\phi_1 \text{ et } q.\phi_2$

**Cas  $\phi = \phi_1 \text{ or } \phi_2$**  Alors Faire **marquage** ( $\phi_1$ );  
     **marquage** ( $\phi_2$ );  
     Pour tout  $q$  dans  $Q$  Faire  $q.\phi := q.\phi_1 \text{ ou } q.\phi_2$

**Cas  $\phi = E \ X \ \psi$**  Alors Faire **marquage** ( $\psi$ );  
     Pour tout  $q$  dans  $Q$   
     Faire Si il existe une transition ( $q, q'$ ) tq  
          $q'.\psi = \text{true}$   
     Alors Faire  $q.\phi := \text{true}$   
     Sinon Faire  $q.\phi := \text{false}$

**Cas  $\phi = A \ X \ \psi$**  Alors Faire **marquage** ( $\psi$ );  
     Pour tout  $q$  dans  $Q$   
     Faire Si il existe une transition ( $q, q'$ ) tq  
          $q'.\psi = \text{false}$   
     Alors Faire  $q.\phi := \text{false}$   
     Sinon Faire  $q.\phi := \text{true}$

**Cas  $\phi = E \ \phi_1 \ \phi_2$**  Alors Faire  
     % on utilisera une liste  $L$  des états pour lesquels  $\phi$  est vraie,  
     % construite en parcourant à l'envers sans boucler les transitions  
     de  
     % l'automate.  
     **marquage** ( $\phi_1$ ); **marquage** ( $\phi_2$ );  $L := \text{vide}$ ;  
     Pour tout  $q$  dans  $Q$  Faire  $q.\text{marque} := \text{false}$  ;  $q.\phi := \text{false}$ .  
     Pour tout  $q$  dans  $Q$  Faire Si  $q.\phi_2$  Alors Faire  $L := L \cup \{q\}$   
     Tant que  $L \neq \text{vide}$  Faire  $q := \text{choix}(L)$ ;  $L := L \setminus \{q\}$ ;  $q.\phi := \text{true}$ ;  
         Pour toute transition ( $q', q$ )  
         Faire Si non  $q'.$ marque Alors  
             Faire  $q'.$ marque := true;  
             Si  $q'.$  $\phi_1$  alors  
                 Faire  $L := L \cup \{q'\}$

**Cas  $\phi = A \ \phi_1 \ \phi_2$**  Alors Faire  
     % on notera par  $\text{degré}(q)$  le le nombre de successeurs de l'état  $q$   
     dans  
     % l'automate. On utilisera un tableau  $q.\text{nb}$  pour parcourir tous les  
     % successeurs d'un état.  
     **marquage** ( $\phi_1$ ); **marquage** ( $\phi_2$ );  $L := \text{vide}$ ;  
     Pour tout  $q$  dans  $Q$  Faire  $q.\text{nb} := \text{degré}(q)$  ;  $q.\phi := \text{false}$ .  
     Pour tout  $q$  dans  $Q$  Faire Si  $q.\phi_2$  Alors Faire  $L := L \cup \{q\}$   
     Tant que  $L \neq \text{vide}$  Faire  $q := \text{choix}(L)$ ;  $L := L \setminus \{q\}$ ;  $q.\phi := \text{true}$ ;  
         Pour toute transition ( $q', q$ )  
         Faire  $q'.$ nb :=  $q'.$ nb - 1  
         Si  $q'.$ nb = 0 et Non  $q'.$  $\phi_1$  et  $q'.$  $\phi_1$  Alors  
             Faire  $L := L \cup \{q'\}$

## Les transformations :

$$A p = \neg E \neg p \quad \text{et} \quad Fp = \text{true } Up \quad \text{et} \quad Gp = \neg F \neg p$$

- $E P \equiv \neg A \neg P$

## Types de propriétés

- **Sûreté (Safety) :**
  - « quelque chose de mauvais **ne doit jamais arriver** »
- **Vivacité (Liveness) :**
  - « quelque chose de bon **finira par arriver** »
- **Équité (Fairness) :**
  - Assure une **justice** dans l'accès à une ressource ou un service.

MAGUEMOUN SAMY